



METAL SYSTEM TRACE + GPU COUNTERS · APPLE M1 MAX · 2026-08-04 · COMMIT **A3ADCA9**

Anatomy of one rendered frame

hero-garden-hose · 405.6 ms

A pass-by-pass account of what the sparse-voxel renderer does to produce the frame above: what each of the thirteen GPU passes computes, which structure it reads, and what the hardware actually does while it runs. The default presentation path — rasterized brick proxies for primary visibility, cone-traced visibility for lighting, deferred relighting — and, at length, the analytic signed-distance solids that are the set and are also the cost. Every number is measured or read out of the code. Nothing is modelled except where marked **DERIVED**.

frame **405.6 ms** · **2.47 fps**

resolution **1600 × 1240**

primary **raster brick proxies**

lighting **cones @ 800 × 620** · **full-res relight**

SDF records **501** · **208 clusters**

octree **584 leaves** · **depth 4**

node-mip pages **245**

frames analysed **33 of 38**

Printed from `docs/svo-render-frame-anatomy.html`, where §3, §5.3, §5.4 and both heatmaps are live: the frame ribbon and timeline are hoverable, the resolve-order figure switches between three real pixels and between today's shading and a front-to-back resolve, the march figure switches cluster field and fold width, and the two maps read out their own per-pixel numbers under the cursor.

THE FRAME — 13 LABELLED METAL ENCODERS, WIDTH $\propto$ GPU TIME



exact live-scene primitive visibility · 347.12 ms · 85.6 % of frame · Fragment channel · frag occ 21.1 % · ALU 22.4 %

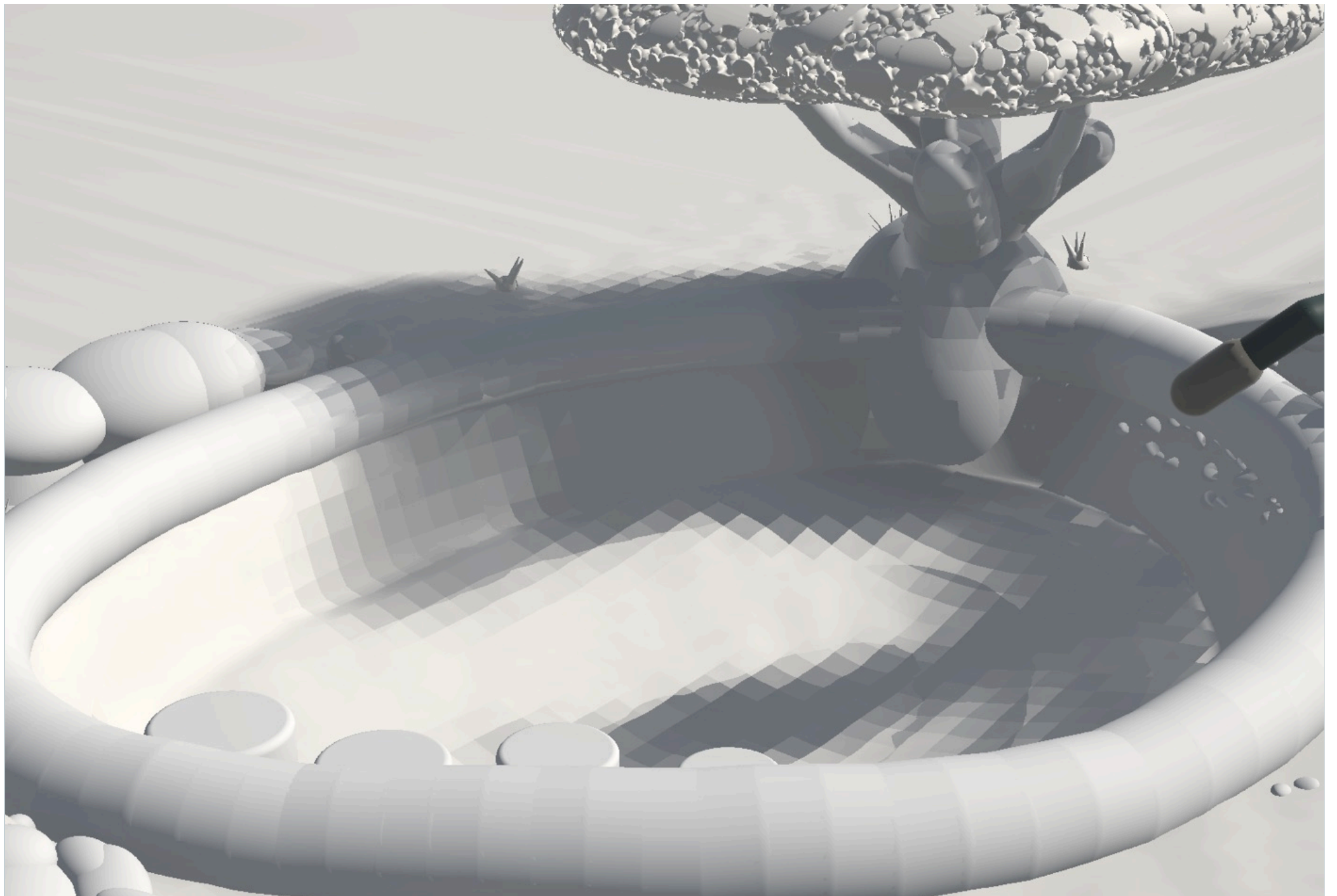
THE THESIS

This frame is not bound by voxel traversal, by cone tracing, or by pixels. It is bound by **one pass that shades every object standing in front of every other object**. `Sparse voxel exact live-scene primitive visibility` — the pass that resolves the scene's 501 analytic signed-distance solids — takes **347.1 ms of a 405.6 ms frame**. Everything else the renderer does, including the whole sparse-voxel octree primary and the entire lighting pipeline, adds up to **58.5 ms**.

The cause is a two-line property of its fragment shader. It returns `@builtin(frag_depth)` and it calls `discard`. Either one alone tells a tile-based GPU that a fragment's depth and coverage are unknown until the fragment has been shaded, so hidden-surface removal cannot run and every proxy box covering a pixel evaluates its own signed-distance field. A covered pixel here sits under a median of 7 such proxies and a 99th percentile of 33, and 95 % of the scene's primitives resolve their hit by a bounded sphere trace rather than a closed-form root.

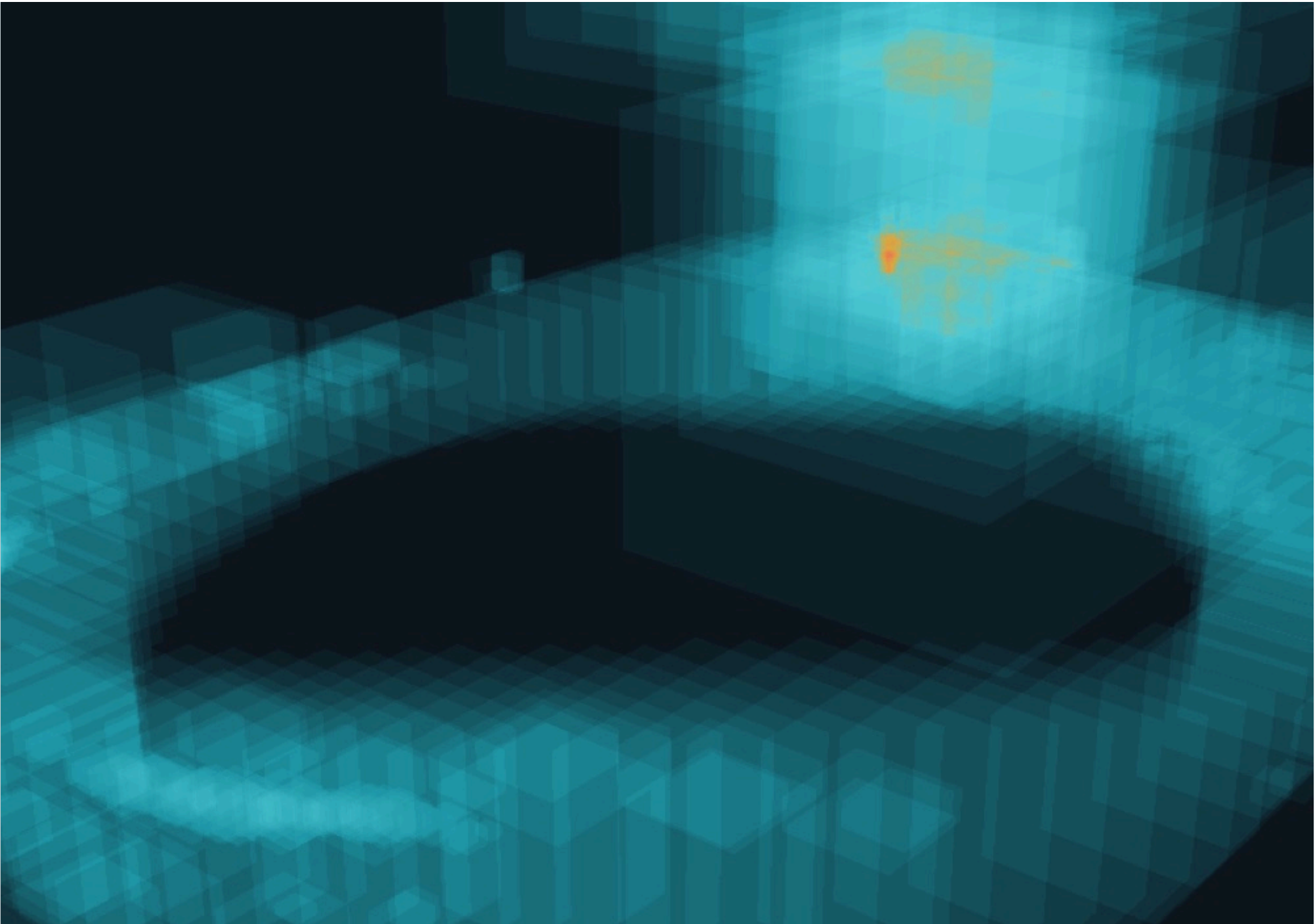
Dropping both — a timing probe whose image is wrong, but which shows what the hardware is willing to reject — takes the frame from **412 ms to 131 ms**, a paired 3.15×. That is an over-statement, and §5.3 measures by how much: a *sound* front-to-back resolve still has to march 5.22 proxies per covered pixel rather than 9.11, so the ordering itself is worth 1.75×. The pass immediately before this one in the same command buffer already contains that design, and costs 3.9 ms.

And the cost is not spread evenly over the set. The **bonsai is 74 of the 501 records** — 15 % — but it accounts for **41 %** of all covering-proxy samples and, once each record's field is priced by what its shader loop actually folds over, for **65 %** of the frame's distance-function evaluations. Section 5 is about that.





The frame. 1600×1240 , raster primary visibility, cone-traced shadows / AO / GI at 800×620 , full-resolution relight, ACES. One warm directional key at intensity 3.2 over a cool sky fill; everything in the set is porcelain-white, which is what makes the visibility term carry the image.



What that frame costs — part one: Covering scene-primitive proxy boxes per pixel, from casting the exact primary ray for every pixel against every primitive's conservative world AABB — precisely the box the vertex stage emits. Dark = 0, teal ≈ 15, amber / ember ≥ 40, peak 59 in the bonsai canopy.

THE WALL

347.1 ms

85.6 % of the frame in one fragment shader. Range over 33 frames 219.8–439.1 ms.

EVERYTHING ELSE

58.5 ms

Octree primary 3.9 ms, rigid + glass 0.27 ms, lighting visibility 46.9 ms, deferred shading 10.7 ms.

RECOVERABLE

3.15×

Paired against the hidden-surface-removal probe: 281 ms median delta over three interleaved pairs.

BONSAI SHARE

65 %

Of all modelled distance-function evaluations, from 15 % of the records. DERIVED

Provenance and measurement integrity

Machine	Apple M1 Max — 4 GPU partitions, 98,304 resident threads
OS / backend	macOS 26.5.1 (25F80) · Dawn/Metal 3 · skip_validation, use_user_defined_labels_in_backend · headless
Lane	tools/profile-svo-render-xctrace.ts --scene=hero-garden-hose --resolution=1600x1240
Configuration	raster-primary · split shading · cones at scale 0.5 · full-res-relight
Source	a3adca9, render fingerprint d842739c0f0f over 69 files. The working tree carried two unrelated solver-side edits; nothing on the render path.
Frames	38 complete, 33 analysed (4 warm-up frames and 1 partial dropped)
Frame wall	405.6 ms median · 281–501 ms
GPU busy	402.5 ms = 99.1 % device utilisation
Untraced baseline	394.2 / 411.9 / 434.0 ms over three interleaved clean runs

PROFILING DISTORTION

0.98x

Traced and label-isolated wall 405.6 ms against a clean untraced median of 411.9 ms. Metal System Trace and per-pass encoder isolation are inside the machine's own frame-to-frame spread on this lane.

ATTRIBUTION

100 %

13 labelled encoders and 6 boundary blits (74 μs in total), nothing unnamed. Per-pass GPU time sums to 408.9 ms against 402.5 ms of device-busy time in the same interval; the difference is genuine Vertex / Fragment / Compute overlap, counted once in busy and once per channel in the table.

9.7 %

GPU hardware counters are device-wide and cannot be filtered by process. Only samples taken while no other process had GPU work in flight are used. Per-pass *time* is exact regardless — it filters by process. Occupancy, ALU and bandwidth are directional.

Four caveats that matter when reading everything below.

1. **Label isolation is free here.** The traced run splits the frame's WebGPU passes into individually-labelled Metal encoders so each carries its own GPU timestamp. It measured **405.6 ms/frame** against a clean, untraced, un-isolated run's **394.2 / 411.9 / 434.0 ms** over three interleaved samples. Per-pass attribution therefore describes the frame that ships, not a distorted one.
2. **Vertex, Fragment and Compute are separate hardware channels and they overlap.** Apple's tiler runs one pass's fragment work against the next pass's binning. Every row of the pass table is one channel of one encoder; summing all rows slightly exceeds the frame wall, and the union — device busy — does not.
3. **The first four frames are excluded.** They carry Metal's first-use residency work: the glass pass's vertex stage alone reports 225–308 ms on frames 1–4 and 20–50 μ s thereafter. 33 steady-state frames are analysed out of 38 complete ones.
4. **The machine was shared.** WindowServer (15.2 s), Codex (Service) (10.9 s), Cursor Helper (4.9 s) held GPU work inside the capture window, which is why counter coverage is low and why the frame wall spreads 281–501 ms. Every timing conclusion below is interleaved and paired for exactly that reason.

§ 2

The scene, as the renderer sees it

`hero-garden-hose` is a dry scene: `systems.fluid` is false, no solver is constructed, and `WebGPULiveSvoScene` stands in for one — it owns an `OctreeSparseBrickWorld` and implements the narrow source interface the renderer attaches to, with no transport, projection or level-set authority at all (`lib/webgpu-live-svo-scene.ts:61`). Every millisecond below is therefore render cost, with nothing simulated behind it.

The set is a raised porcelain pond. Its vessel — coping crest, basin, and the ground outside it — is *one generated heightfield* rather than geometry, so the water would meet the rim exactly where the rim is. Everything standing on that heightfield is an analytic signed-distance solid, and there is no triangle mesh anywhere in the frame.

DOMAIN AND LATTICE	
Container	1.8 × 0.6 × 1.2 m, open top
Finest cell	25 mm · brick 8 ³ cells
Render lattice	72 × 24 × 48 cells
Octree depth	4
Camera	azimuth 5.40 rad, elevation 0.40 rad, 1.4 m out
Lens	tanHalfFov 0.24 — a 50 mm on 35 mm, authored, not the 0.72 default
Lights	1 directional key (warm, intensity 3.2) + analytic sky fill
Rigid bodies	5 stepping stones
Glass panes	0
Fluid	none — <code>systems.fluid</code> false

PUBLISHED TO THE GPU

Octree topology	12,534 nodes, 584 leaves — 0.39 MB
Brick payload	299,008 voxels $\times$ (geometry + velocity + owner) — 10.27 MB
Compact hierarchy	245 hot nodes at 16 B — 50 % narrower than canonical
Node-mip pyramid	245 pages, $8^3 + 1$ apron, 4 channels — 1.03 MB
Tetrahedral radiance	245 pages $\times$ 4 lobes, 0 black — 14.56 MB
Wide fan-out	0.01 MB
SDF records	501×64 B = 31 KB, plus 192 KB of cluster arena blocks
Terrain	analytic heightfield, sampled at 6.25 mm — a quarter of a cell
Total resident	36.66 MB

Two entries in the right-hand column are worth pausing on, because they are what the lighting passes in §4 actually read.

The **node-mip opacity pyramid** is a sparse, page-backed mip chain over the octree: 8^3 interiors with a one-voxel apron, four channels (solid mean, solid max, fluid mean, fluid max), 4 000 B per page, 245 pages resident here. It is what a cone samples instead of a ray marching leaves. Publication is all-or-nothing by design — a truncated pyramid leaks light rather than coarsening, so `OctreeSparseBrickWorld` withdraws the whole thing if any base page is omitted, and cone lighting silently falls back to exact traversal for every shadow and GI ray. That failure is a $15\times$ render regression that reads as a renderer bug; on this scene it published cleanly.

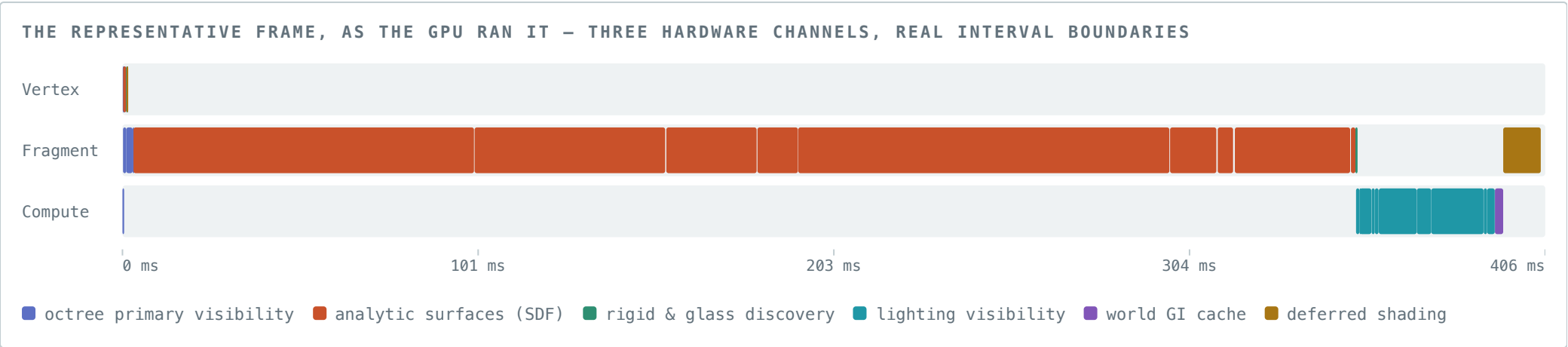
The **tetrahedral radiance atlas** is four directional lobes of outgoing radiance over the same pages, 14.56 MB — the cached bounce the GI cones gather rather than re-trace.

§ 3

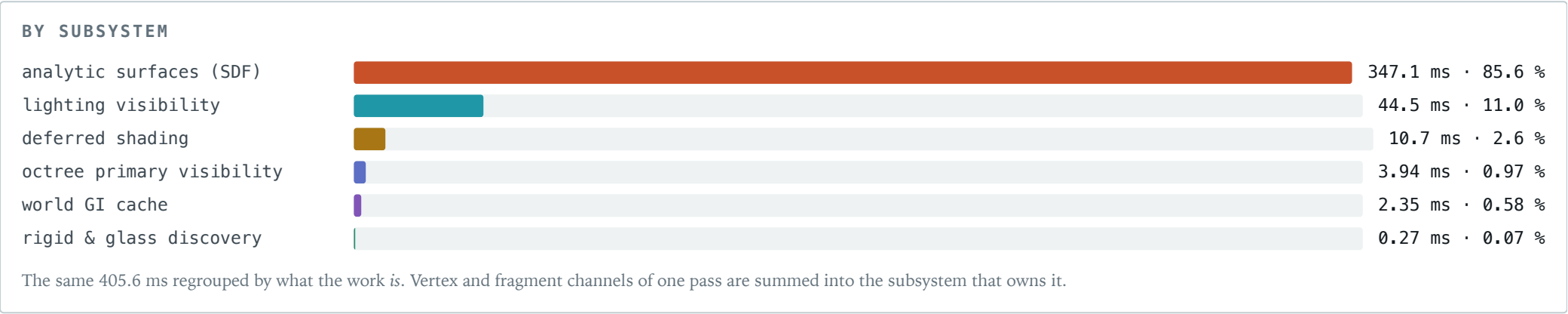
The frame at a glance

One frame is one command buffer and one submission. Thirteen labelled passes are encoded in a fixed order by `SparseVoxelDrySceneRenderer.encode ( lib/webgpu-svo-dry-scene.ts:7178 )`, plus six boundary blits that

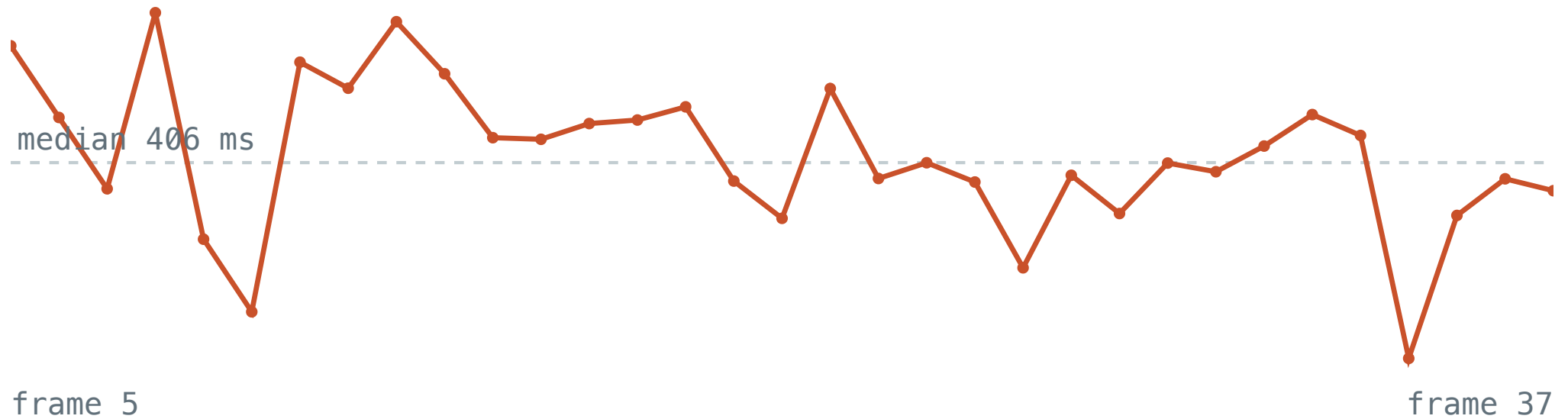
together cost 74 μ s. There is no readback and no host decision anywhere inside the frame: the only indirect arguments are the brick instance count the cull pass publishes for itself.



Every rectangle is one Metal GPU interval from the trace, placed at its own start and drawn at its own duration. The frame shown is the one whose anchor-to-anchor wall is closest to the 33-frame median. Note that the scene-primitive pass arrives as several intervals — Instruments splits a long encoder wherever the GPU preempted it — and that all thirteen vertex stages have finished binning inside the first 3 ms.



WHICH NUMBERS ARE STABLE – 33 FRAMES



The frame wall spreads **281–501 ms** and essentially all of that spread is one pass: the scene-primitive raster alone ranges **219.8–439.1 ms**, while cone visibility holds **35.3–48.9**, deferred lighting **10.5–11.5** and the whole octree primary inside $\pm 6\%$. On a machine shared with a compositor and a browser, the pass that owns 85.6 % of the frame is also the one that absorbs the contention.

§ 4

Pass by pass

Timings are the 33-frame median. Occupancy, ALU, bandwidth and last-level-cache utilisation are device-wide hardware counters attributed to whichever pass owned the GPU when the sample was taken, from uncontended

windows only. **Fragment Occupancy is the counter this lane samples**, so it reads near zero for the compute passes by construction rather than by starvation.

P1 brick instance cull

Compute 0.028 ms · 0.02–0.05 ms across frames

0.03

ms · 0.01 %

Emit one box instance per resident, occupied SVO leaf brick, frustum-cull it, and bucket-sort it by minimum view depth.

Three dispatches in one encoder — emit, scan, scatter — over the leaf capacity, publishing an indirect draw for the passes below. The box is not the leaf: it is the leaf's *occupied sub-AABB*, decoded from the occupancy word the sparse-brick producer already writes into the terminal node's `links.w`. Sub-boxes of disjoint boxes stay disjoint, so a ray still meets each instance over one interval and those intervals stay totally ordered.

The sort is a 1024-bucket counting sort and is deliberately approximate inside a bucket: the depth test is the authority and the ordering exists only so tile-based hidden-surface removal gets to reject occluded bricks before shading them. That is the same mechanism §6 is about, used correctly.

frag occ 10.7%

ALU 10.0%

LLC 27.0%

read 17.4 GB/s

write 41.3 GB/s

P2 primary conservative brick coverage

Fragment 0.906 ms · Vertex 0.041 ms · 0.86–1.00 ms across frames

0.95

ms · 0.23 %

Append every brick proxy covering a pixel into a fixed 24-entry per-pixel arena. No depth, no payload, no shading.

One `atomicAdd` per covering fragment and a store. Its `discard` follows the append and only suppresses the throwaway overflow-mask colour write, so it costs nothing that matters. This is the cheapest possible way to learn what covers what.

The counters say exactly that: high fragment occupancy at almost no ALU, with tens of GB/s of writes. The machine is full and doing nothing but memory traffic — the correct shape for this pass.

frag occ 64.7%

ALU 3.0%

LLC 27.1%

read 0.3 GB/s

write 47.8 GB/s

P3 primary conservative coverage resolve

Fragment 1.83 ms · Vertex 0.047 ms · 1.82–2.09 ms across frames

1.88

ms · 0.46 %

One expensive fragment per pixel walks that pixel's candidate list front-to-back and writes the exact hit or an exact miss.

A full-screen triangle. Every pixel resolves once regardless of how many bricks covered it, and it always writes — hit or miss — so it never discards. Terrain and the sky background are folded in here too.

Its counters are the mirror image of the coverage pass: high ALU at moderate occupancy and very little bandwidth. This is the one pass in the frame that is ALU-bound in a healthy way.

frag occ 18.5%

ALU 40.8%

LLC 5.4%

read 2.0 GB/s

write 12.8 GB/s

P4 primary conservative coverage overflow

Vertex 0.841 ms · Fragment 0.243 ms · 0.65–0.93 ms across frames

1.08

ms · 0.27 %

Re-run the historical one-DDA-per-proxy brick fragment, but only for pixels whose candidate list overflowed 24 entries.

Capacity never changes the image; it only decides how much work lands in this isolated pipeline. Garden's measured maximum was 18 against the arena's 24, and on this scene the pass costs a quarter of a millisecond of fragment time — the overflow set is effectively empty.

frag occ 64.6%

ALU 9.4%

LLC 31.0%

read 39.6 GB/s

write 48.6 GB/s

P5 exact live-scene primitive visibility

Fragment 346.22 ms · Vertex 0.901 ms · 219.84–439.11 ms across frames

347.1

ms · 85.6 %

Draw one conservative box per authored signed-distance primitive and resolve the exact surface in the fragment.

One instance per record, 36 vertices each, back faces only so a camera inside a proxy still shades it. A proxy that contains the eye emits a full-screen triangle instead. The fragment rebuilds the primary ray from the pixel, brackets the march interval to the box span, and either returns the exact surface or discards.

It is depth-tested against the brick raster's buffer, which already holds the octree's answer — and it cannot use it, because the fragment writes `frag_depth` and discards. §5 and §6 are this pass.

Low fragment occupancy at moderate ALU and almost no bandwidth: long shaders with big register footprints and no memory pressure. The machine is a quarter busy computing and largely empty of threads.

frag occ 21.1%

ALU 22.4%

LLC 8.5%

read 1.2 GB/s

write 11.2 GB/s

P6 analytic rigid primary discovery

Fragment 0.108 ms · Vertex 0.070 ms · 0.09–0.41 ms across frames

0.18

ms · 0.04 %

Rasterize the stepping stones as analytic impostors into the same G-buffer.

Bodies are exact quadrics rather than voxels, so they get their own proxy pass and their own certificate plane. The adaptive crossover between this and tracing them inside the megakernel picks raster here.

frag occ 22.9%

ALU 18.2%

LLC 23.2%

read 18.3 GB/s

write 40.6 GB/s

P7 raster-rigid certificate bridge

Fragment 0.036 ms · Vertex 0.029 ms · 0.03–0.12 ms across frames

0.07

ms · 0.02 %

Copy the rigid pass's private geometry plane into the split G-buffer's geometry and identity planes.

A pure plumbing pass, and it exists so the rigid impostor's exact normal and identity survive into deferred shading without the primary pass having to know about bodies.

frag occ 39.6%

ALU 44.3%

LLC 10.9%

read 16.8 GB/s

write 18.3 GB/s

P8 raster thin-glass discovery

Vertex 0.029 ms · 0.02–0.05 ms across frames

0.03

ms · 0.01 %

Rasterize thin glass panes into a key plane, coverage-scaled and depth-tested separately.

This scene has no glass at all — `glassPaneCount` is 0 — so the pass encodes a draw of zero instances and produces no fragment work whatsoever. Its microsecond-scale vertex stage is the entire cost of keeping the path uniform across scenes.

frag occ 12.9%

ALU 32.2%

LLC 3.1%

read 0.2 GB/s

write 0.8 GB/s

P9 directional-light cache demand

Compute 1.16 ms · 0.90–1.56 ms across frames

1.16

ms · 0.29 %

Classify every full-resolution pixel against the voxel directional-light cache and enqueue the cache misses.

An 8×8 compute grid over the full frame. Where a world-space cache entry already answers the key light's visibility for a voxel and normal, the cone march below is skipped entirely.

frag occ 5.3%

ALU 12.0%

LLC 12.5%

read 14.7 GB/s

write 1.0 GB/s

P10 directional-light cache population

Compute 0.010 ms · 0.01–0.15 ms across frames

0.01

ms · 0.00 %

Fill the queued cache entries under a fixed per-frame budget.

A bounded 64-lane dispatch — ten microseconds. The budget is the point: the cache converges over frames rather than stalling one.

frag occ 0.0%

ALU 0.0%

LLC 0.5%

read 0.1 GB/s

write 0.1 GB/s

P11 compact cone visibility

Compute 43.37 ms · 35.28–48.86 ms across frames

43.4

ms · 10.7 %

Resolve shadow, ambient-occlusion and global-illumination visibility for every reduced-rate pixel by marching cones through the node-mip opacity pyramid.

Three dispatches in one encoder: a single-lane reset of the boundary queue, the 8×8 coherent march over the reduced plane, and a 64-lane pass that retraces exactly the pixels whose 2×2 full-resolution footprint was not homogeneous.

Packing is exact rather than approximate: `rg32uint` holds 8-bit AO plus eight 7-bit light visibilities in 64 bits, so a scene may shade up to eight lights through one plane. This one has a single directional key.

Fragment occupancy reads zero because this is a compute pass and Fragment Occupancy is the counter this lane samples — not a starvation reading.

frag occ 0.4%

ALU 16.4%

LLC 9.7%

read 1.2 GB/s

write 1.1 GB/s

P12 persistent world GI cache

Compute 2.35 ms · 2.34–2.86 ms across frames

2.35
ms · 0.58 %

Gather one bounce of diffuse radiance from the tetrahedral radiance atlas into a persistent world-space hash cache.

The cache is keyed in world space and owned by the source, so camera motion changes which keys are queried but never invalidates an entry. Only a source, authored-scene or lighting-contract change clears it — which is why it is flat to a fraction of a millisecond across every frame in the window.

The highest ALU utilisation in the frame. A dense, well-shaped kernel.

frag occ 0.1%

ALU 58.0%

LLC 17.5%

read 2.1 GB/s

write 0.7 GB/s

P13 deferred dry lighting

Fragment 10.66 ms · Vertex 0.017 ms · 10.54–11.49 ms across frames

10.7
ms · 2.6 %

Evaluate materials, BRDF and the tone curve at full resolution against the reduced visibility plane.

Two draws in one pass, partitioned by two complementary depth tests: the sky shader takes the pixels primary visibility left as a miss, and the full deferred shader takes the rest. Sky goes first so the expensive draw is the one still in flight when the pass ends. The depth attachment is `depthReadOnly`, so it costs a tile load and no store.

This is where `full-res-relight` is paid: material and BRDF are re-evaluated per pixel, and only visibility comes from the half-rate plane. Tens of GB/s of writes at low occupancy — bandwidth-shaped, as a deferred resolve should be.

frag occ 12.0%

ALU 10.9%

LLC 28.8%

read 20.2 GB/s

write 45.7 GB/s

The signed-distance set, in detail

The set is not geometry. Every object in the frame except the terrain is one 64-byte record naming an analytic solid, and the renderer resolves each one exactly, per pixel, in a fragment shader. That is what makes the scene cheap to author, cheap to store — 501 records is 31 KB — and free of any level-of-detail decision: a floret is silhouetted exactly at any distance, and the same canopy rendered at 25 mm, 50 mm and 100 mm voxels produces three byte-identical frames, because a scenery primitive's primary visibility never goes through the voxel payload at all.

It is also, at this scale, 85.6 % of the frame. This section is about where inside the set that cost actually sits, which is not where the pixels are.

5.1 What one marched hit costs

A kind is declared once, in `lib/svo-primitive-kinds.ts`: u32 tag, flags, normal policy, what the three dimension floats mean, the conservative local extent and the rotation-invariant bounding radius. Every consumer keys an exhaustive `Record<SvoPrimitiveKindName, T>` off it, so a kind cannot be added to one path and be silently missing from another — which is the failure that table exists to prevent, because an unknown kind used to fall through to an ellipsoid in the node-mip oracle, to `1e20` in the voxelizer, and to a vertex-shader bail in this very raster.

The flag that decides cost is `marchedIntersection`. A kind carrying it has no closed-form root: its hit is a bounded sphere trace of its own distance function, capped at `SVO_PRIMITIVE_MARCH_ITERATIONS = 48`, over the interval the proxy box brackets. **476 of 501 primitives here are marched.**

And a march step is not one distance call. For a `smooth-union-cluster` — the aggregate kind, 208 of them in this scene — a step evaluates a whole procedural *field*, and the field is a fold over lobes, segments or lattice cells. Then the shading normal costs four *more* whole field evaluations: a tetrahedral gradient rather than the obvious six-tap central difference, precisely because "an evaluation is roughly sixty sphere distances and the march converges in about eight steps, so the two extra taps would be a fifth of the whole trace" (`lib/svo-primitive-abi.ts:2600`).

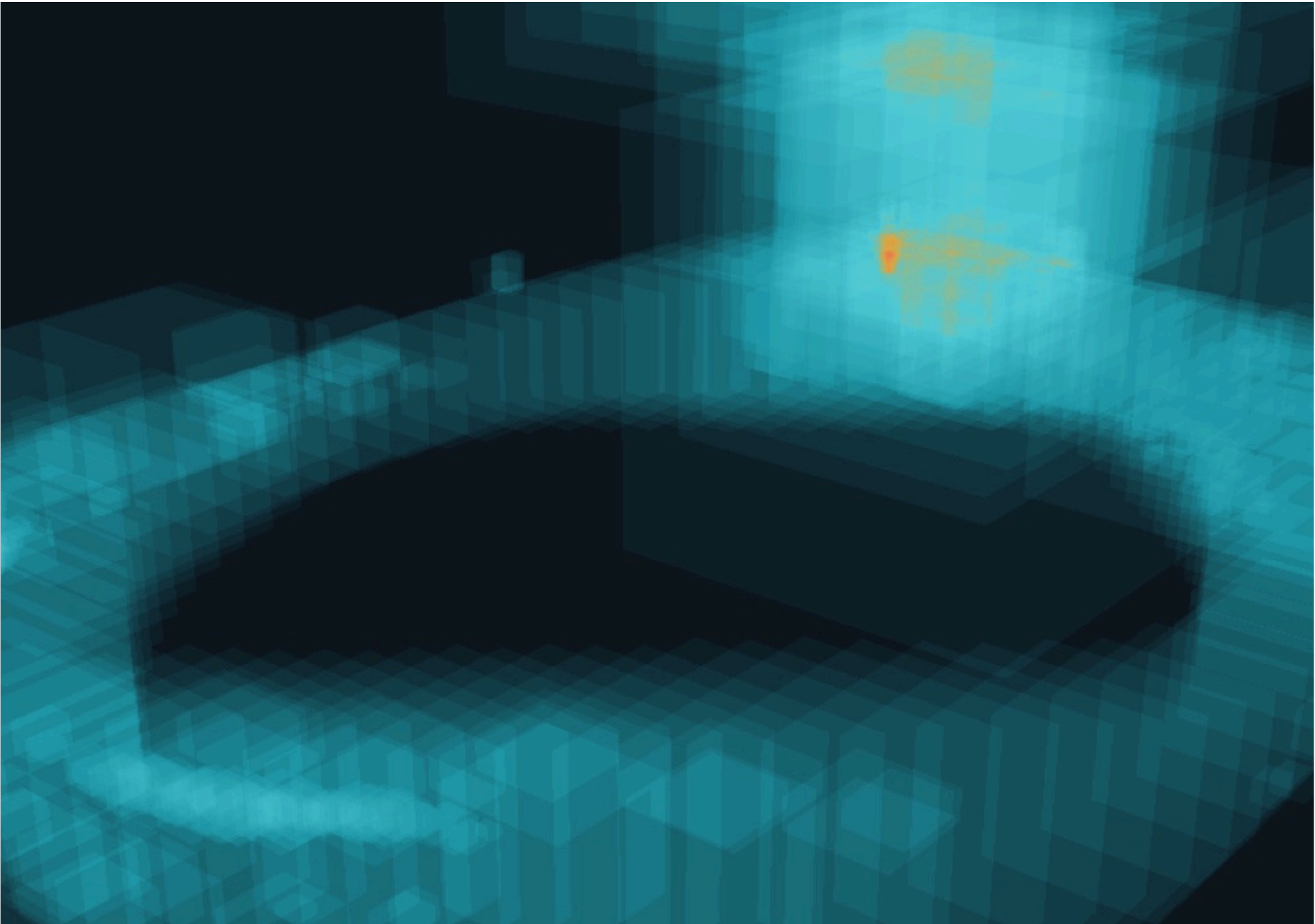
THE THREE CLUSTER FIELDS AND WHAT THEIR LOOP FOLDS OVER

FIELD	RECORDS	FOLD WIDTH	THE LOOP
lattice	148	9.0–72	8 unconditional sphere distances + up to 56 conditional, per octave. The 56 are rejected by an unjittered-centre distance that needs no hash.
tapered-sweep	40	1.5	one exact round-cone distance per polyline segment – the convex hull of two spheres, so Lipschitz-1 by definition.
seeded-lobes	20	6.5	one rotated anisotropic ellipsoid distance per lobe, smooth-min folded in authored order.

lattice is the expensive one and the reason is geometric: a domain-repeated lattice of jittered spheres has to fold over a 4^3 block rather than the obvious 2^3 , because a four-cell block is the smallest one centred on the sample that contains the eight surrounding cells with a ring to spare — and without that ring a sphere leaving the neighbourhood makes the field step, and rays tunnel. The 56 outer cells are rejected by a distance to their *unjittered* centre, which needs no hash, so the realised width is between 8 and 64 per octave.

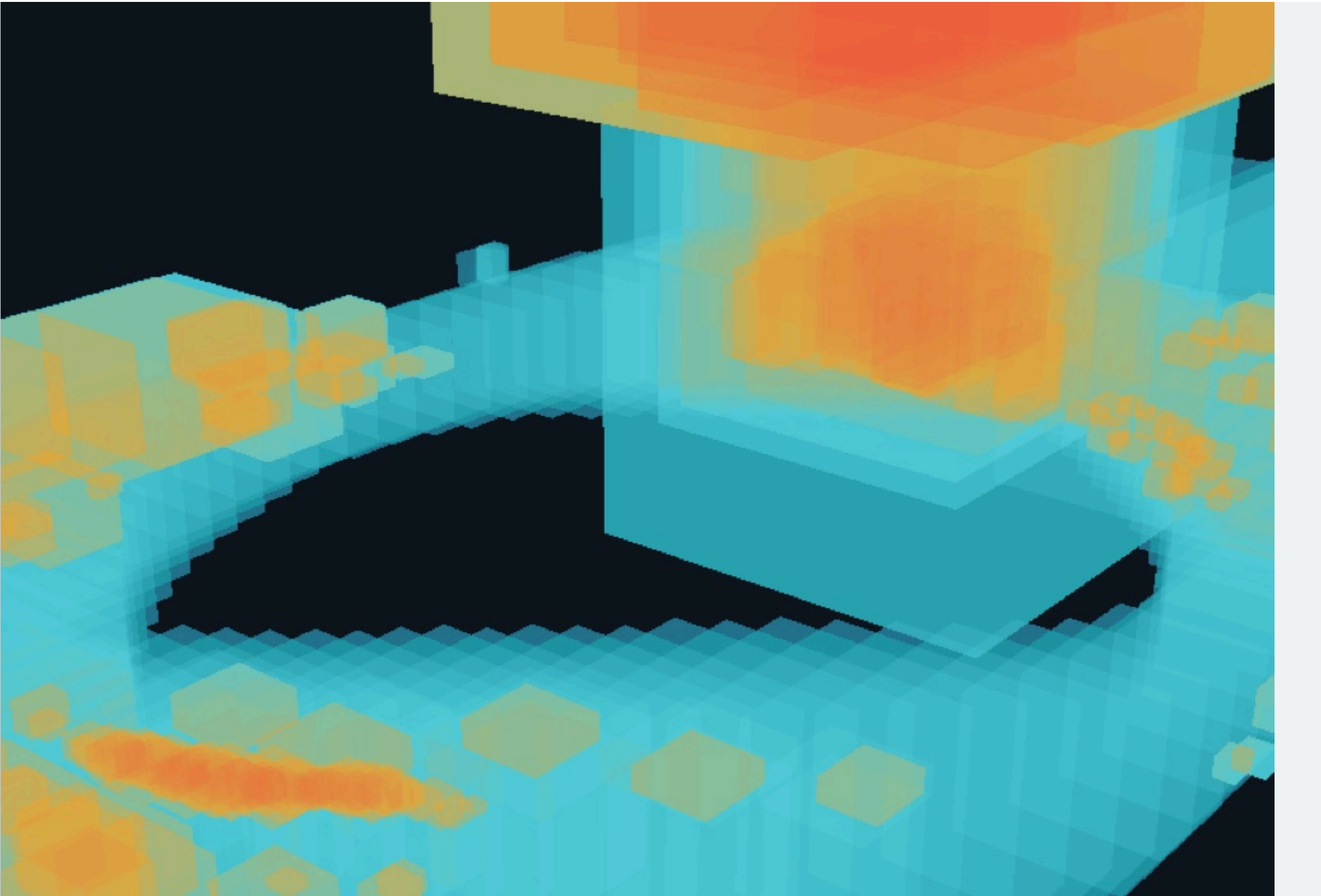
5.2 Coverage is not cost

Two maps of the same frame. The first counts covering proxies per pixel — the number of fragments the missing hidden-surface removal forces this pass to shade. The second weights each of those by what its kind's shader loop folds over, and they do not agree at all.





Covering proxies. The coping rim dominates — a swept bullnose spelled as 84 round cones, laid nose to tail all the way round the pond, so a grazing ray crosses a dozen of their boxes. **46 % of all covering-proxy samples are coping.**





nearly half the coverage.

$\log_{10}(1 + \text{evaluations})$  0 → 1,926

MARCHED PRIMITIVES

476 / 501

95 % of the set resolves its hit by a bounded sphere trace, capped at 48 iterations.

MEAN COVERING PROXIES

8.95

over the 77.8 % of the screen a proxy covers. Median 7, p90 20, p99 33, max 59.

DISTANCE EVALUATIONS

207–994

per covered pixel, floor to ceiling fold width. DERIVED

TIME PER COVERED PIXEL

225 ns

347.1 ms over the 1.54 M covered pixels of the 1600 × 1240 frame.

PART OF THE SET	RECORDS	CLUSTERS	MARCHED	MEAN FOLD	SHARE OF COVERING PROXIES	SHARE OF DISTANCE EVALUATIONS
bonsai	74	65	65	5.5	41.0 %	65.2 %
stones & pebble beds	161	139	153	7.0	10.2 %	17.8 %
coping	84	0	84	1.0	46.0 %	15.9 %
hose	5	4	4	1.2	2.5 %	0.9 %
plants	177	0	170	1.0	0.3 %	0.1 %

Coverage is exact: a ray/AABB slab test of the hero camera's own pixel rays against each primitive's conservative world box, which is the box the vertex stage emits. Evaluations are that same exact coverage multiplied by each record's fold width and by the shader's typical eight-step march — `DERIVED`, and stated as a share rather than an absolute because the step count of a *missing* march is not modelled here. The map's shape does not depend on that constant; only its scale does.

5.3 One pixel, in full

The map above is a count. This is the thing the count is counting. Every proxy one real primary ray enters, in the order a front-to-back resolve would try them, with the *measured* answer to "does this primitive actually get hit" from `intersectSvoPrimitive` — the CPU half of the contract the WGSL intersection is held to at 3e-4 by `tests/webgpu-svo-primitive-exact.test.ts`.



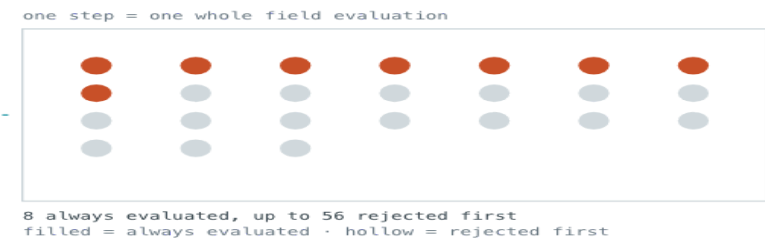
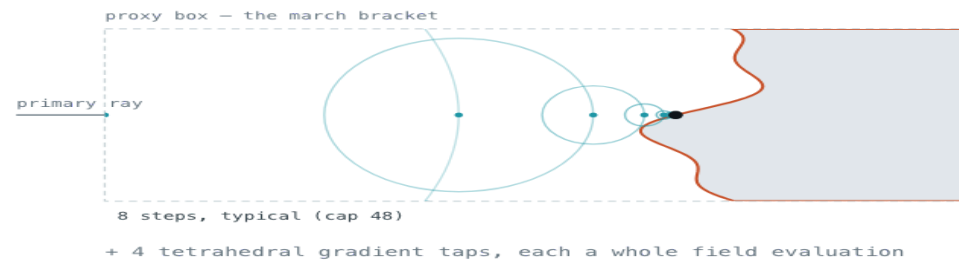
marches 16 of 59 · 43 never reached · 3.69x fewer

■ bonsai ■ coping ■ plants ■ never reached – pure waste

Pixel (540, 154) of the 800×620 census — **the busiest pixel in the frame**. **59** proxies cover it and **16** of them contain a surface the ray actually meets; the nearest is at **1.88 m**, and the vessel heightfield the brick raster already wrote is at **2.16 m**. A resolve that walks entry-sorted and stops once the next candidate starts behind its best hit runs **16** marches. Today all **59** run, because the fragment writes `frag_depth` and discards.

5.4 Inside one march

And this is what one of those bars costs. A marched kind has no closed-form root: the fragment sphere-traces its own distance function inside the proxy's bracket, and for a cluster each step of that trace evaluates a whole procedural field — a fold over lobes, segments or lattice cells. Then the shading normal costs four more whole evaluations.

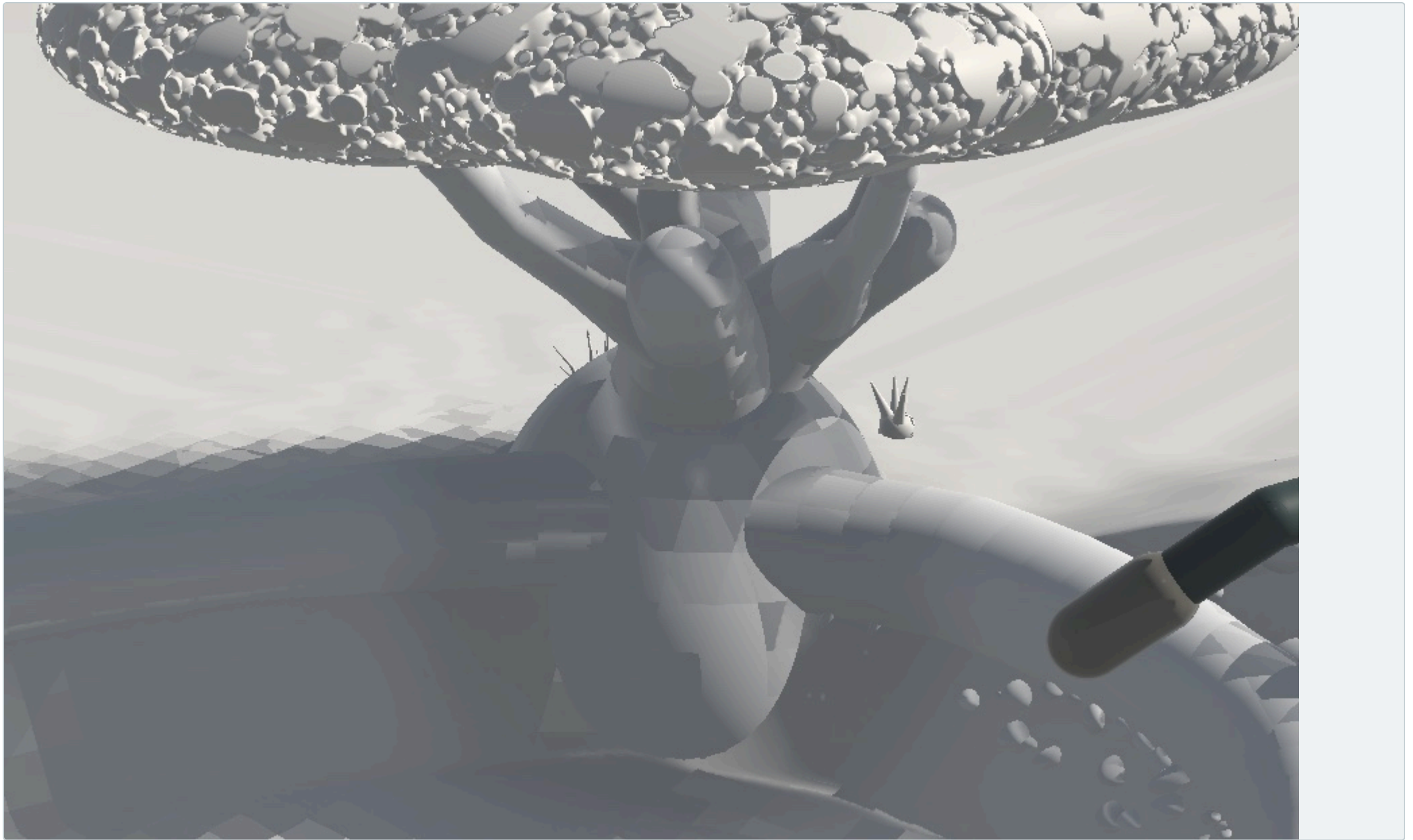


one hit = (8 steps + 4 normal taps) × 24 = 288 distance evaluations

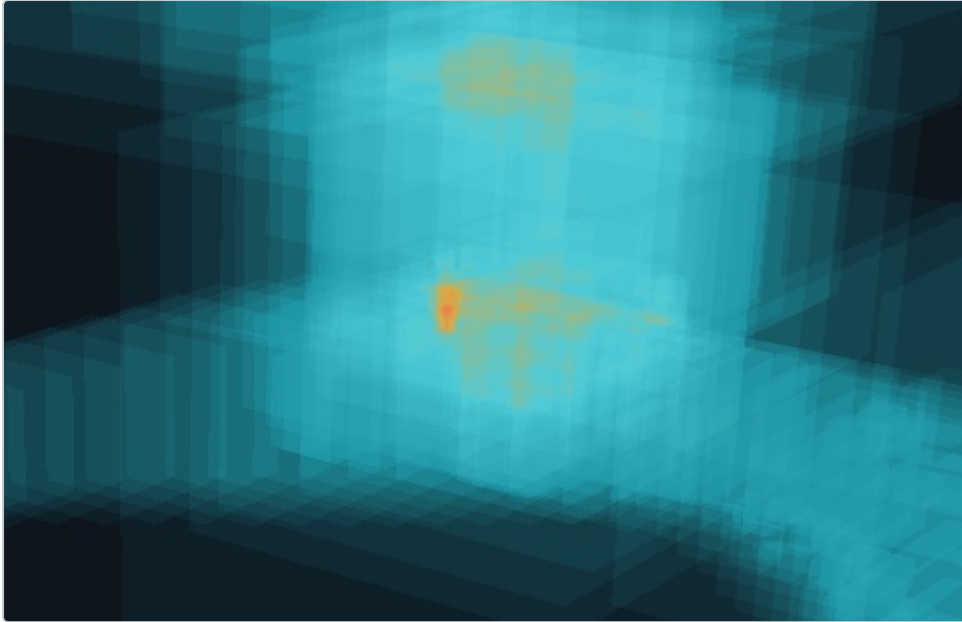
A **lattice** cluster — 8 unconditional sphere distances plus up to 56 rejected by a distance to their unjittered centre, which needs no hash. The 4^3 block is the smallest one centred on the sample that contains the eight surrounding cells with a ring to spare. The bonsai publishes 9 of these, and the scene as a whole 148. At the busiest pixel 59 proxies each run a trace like this one, and a sound resolve would run 16.

5.5 The bonsai

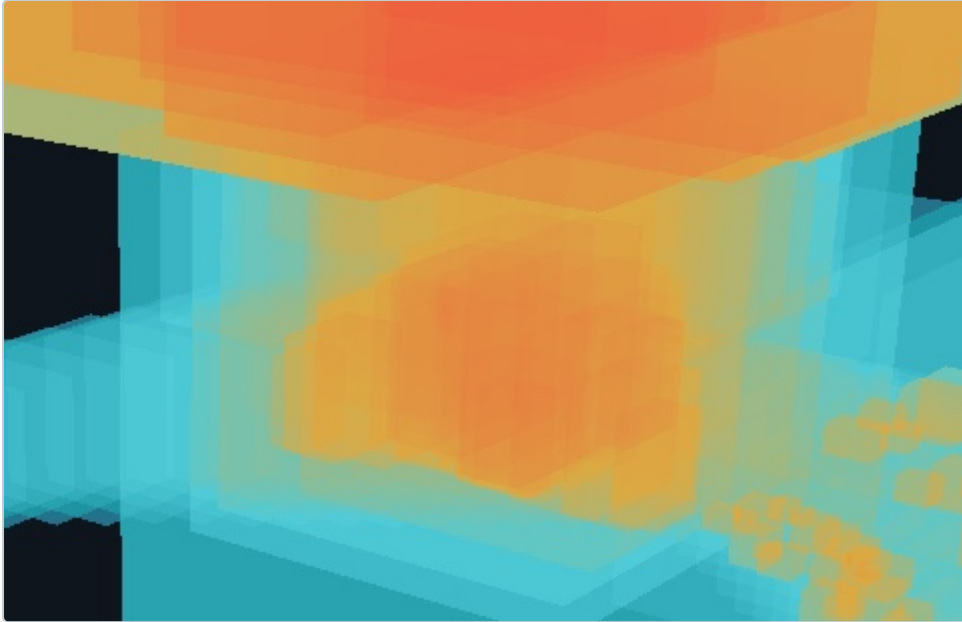
74 records — 15 % of the set — carrying 41 % of the coverage and 65 % of the evaluations. It is worth taking apart, because every decision in it is defensible and the aggregate is still the most expensive object in the frame by a wide margin.



What it is. A fused multi-trunk root mass under one wide crown of cauliflower florets. No sky in the crown, no pads, no tiers.



Its proxies. Peak 59 boxes over one pixel — the crown's cluster envelopes, their opacity cores, and every limb sweep behind them.



Its cost. The hottest region in the frame, and by the model above the single densest concentration of ALU work anywhere in the image.

WHY IT IS BUILT OUT OF CLUSTERS

The crown is one wide plate whose entire character is the floret structure covering it. Spelled explicitly, that is roughly $2 \cdot \text{area} / (\pi \cdot w^2)$ records for a floret of width w — so halving the floret quadruples the bill — against a scene-wide budget of 4 096 primitive-candidate leaves that is a *cliff* rather than a slope: above it the candidate build is simply not called and the whole set stops being drawn. The reference's six-millimetre floret was unreachable with the entire scene budget.

A cluster evaluates its packing in the shader, so its grain costs no records at all and that economy stops applying. What it cannot do is be both granular and closed — a lattice only stops letting rays through at $r \geq \text{period} \cdot \sqrt{3}/2$, and at that ratio the packing is solid, the envelope's hard max cuts it off at exactly the envelope, and the lobe draws as a bare ellipsoid. So the crown is **two records per lobe**: a lattice cluster at a ratio where florets are distinct, and an ellipsoid *core* one granulation-depth under it that no ray gets past.

The trunk and limbs were a chain of cones with a sphere at every node. That construction is exact and it still rendered wrong: a hard union is C^0 but not C^1 , so the shading normal steps at every junction and a smooth taper came out as a stack of faceted pipes. Subdividing does not help, because subdividing adds junctions. A tapered sweep fuses its segments with a smooth minimum — C^1 through the junction — and costs one record for a whole run instead of two per segment.

74 RECORDS, BY WHAT THEY EVALUATE

lattice cluster – the florets		9 records
seeded-lobes cluster – canopy...		20 records
tapered sweep – trunk, buttre...		36 records
ellipsoid – opacity cores		9 records

THE CROWN'S OWN ARITHMETIC DERIVED

```
# one pixel on the canopy, at the census peak
covering proxies          59
of which marched          59

# what one of them costs per march step
lattice, 1 octave, 43 block      8 – 64 sphere distances
seeded-lobes, 5–8 lobes        5 – 8 ellipsoid distances
tapered-sweep, 1–3 segments    1 – 3 round-cone distances

# and the march, and the normal
sphere-trace steps, typical     8      (cap 48)
tetrahedral gradient taps       4      whole field evaluations

# so the winning floret alone, before any of the other 58
(8 + 4) × 8..64          96 – 768 sphere distances
and 58 of the 59 resolve nothing at all.
```

None of this is the bonsai's fault, and that is the point. Every choice above is the cheap one available: the cluster kind exists precisely because the explicit build did not fit in the record budget, the sweep exists because the exact union shaded wrong, and the two-record lobe exists because one record cannot be both granular and opaque. The set is close to minimal for what it draws.

What is not minimal is *how many times each record is asked to draw itself*. The canopy's 65 % share of distance evaluations is a share of work that is largely **thrown away**: at a pixel under 59 proxies, 58 of those field evaluations resolve a surface that something nearer already covered. Making the bonsai cheaper is a much smaller lever than making the pass stop shading what it cannot see — which is \$6.

The wall – hidden-surface removal in the scene-primitive raster

Both raster passes in this frame draw one conservative box per object and resolve the exact surface in the fragment. They are otherwise the same idea. One costs 3.9 ms and the other costs 347.1 ms, and the difference is not the geometry.

THE SCENE-PRIMITIVE FRAGMENT – LIB/WEBGPU-SV0-DRY-SCENE.TS:2613

```
@fragment fn dryScenePrimitiveRasterFragment(
  input: DryScenePrimitiveVertexOut,
) -> DrySceneRasterOut {
  dryRasterPrimaryReset();
  if (input.primitiveIndex >= dry.metadata.x) { discard; }
  let record = dryPrimitive(input.primitiveIndex);
  if (dryOpaqueOwnerSuppressed(...)) { discard; }
  let camera = dryRasterPrimaryCamera();
  let ro = camera[0];
  let rd = dryRasterPrimaryRay(input.position.xy, camera);
  // bounded sphere trace, up to 48 iterations, over a
  // field that may itself fold over 64 lattice cells
  let exact = dryScenePrimitiveExact(record, ro, rd, span);
  if (!(exact.t < DRY_MISS)) { discard; }
  return drySceneRasterOut(dryRasterPrimarySurface(exact, ...));
}

struct DrySceneRasterOut {
  @location(0) packedSurface: vec4u,
  @location(1) identityMedia: vec4u,
  @location(2) geometry:      vec4f,
  @location(3) opaqueIdentity: vec2u,
  @builtin(frag_depth) hardwareDepth: f32,
}
// frag_depth -> depth unknown until shaded
// discard    -> coverage unknown until shaded
// => no hidden-surface removal. Every proxy shaded.
```

```

// Stage one is deliberately coverage-only. Ordinary
// fragments append their candidate and stop immediately;
// only overlaps beyond the fixed arena write the
// throwaway colour. No path traces a payload or writes
// fragment depth.
@fragment fn svoBrickCoverageFragment(
  input: SvoBrickRasterVertexOut,
) -> @location(0) u32 {
  let pixel = coordinate.y * width + coordinate.x;
  let slot = atomicAdd(&svoBrickCoverageCounts[pixel], 1u);
  if (slot < 24u) {
    svoBrickCoverageCandidates[pixel * 24u + slot]
      = input.instanceIndex;
    // the discard here only suppresses the overflow
    // mask, and it follows the append
  }
  ...
}

// then one full-screen fragment per pixel:
resolve.draw(3) // walks 0..count front-to-back, once,
                // always writes an exact hit or miss

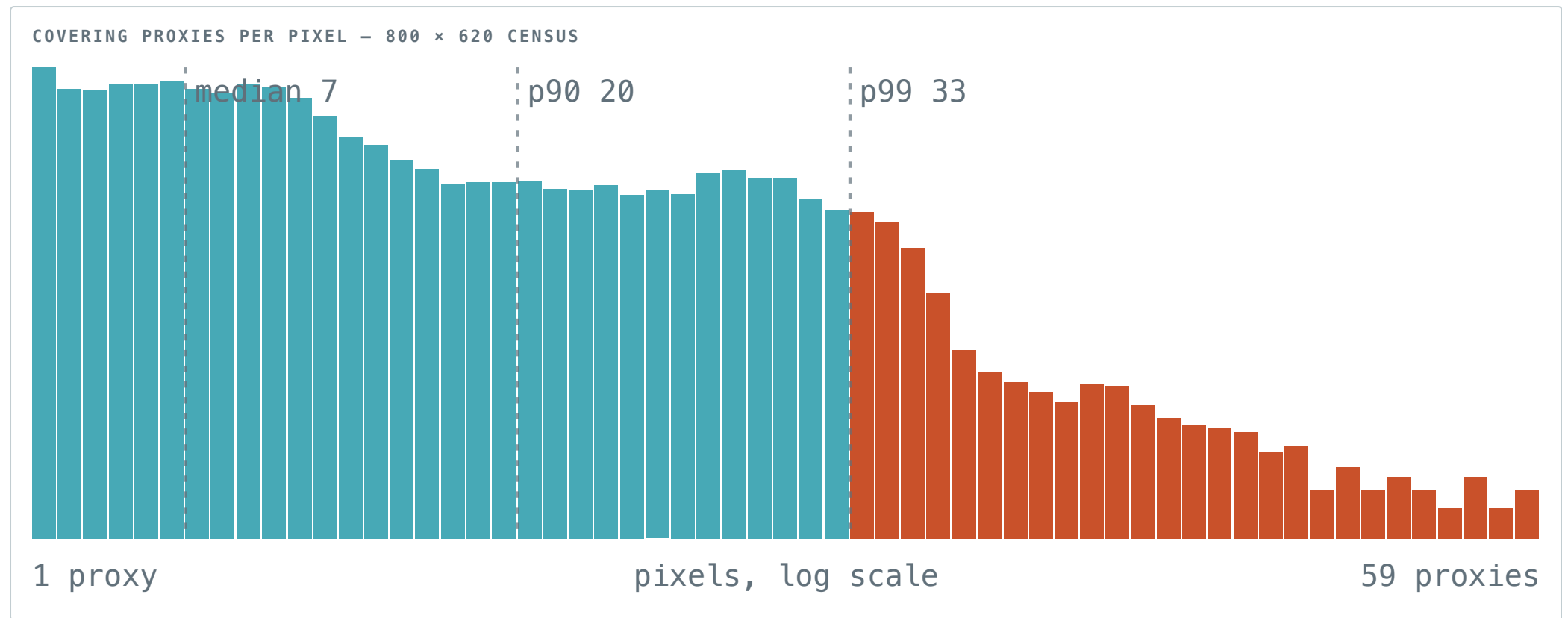
// 0.95 coverage + 1.88 resolve + 1.08 overflow ms

```

Apple's tile-based deferred renderer removes hidden surfaces before shading, but only when it can know a fragment's depth and coverage from the rasterizer alone. A shader that returns `@builtin(frag_depth)` moves the depth decision after shading; a shader that calls `discard` moves the coverage decision after shading. The scene-primitive fragment does both, so its 501 proxies are shaded in submission order with no rejection at all — every covering box marches its own field even when a nearer box already resolved the pixel, and these proxies, unlike brick proxies, are not disjoint.

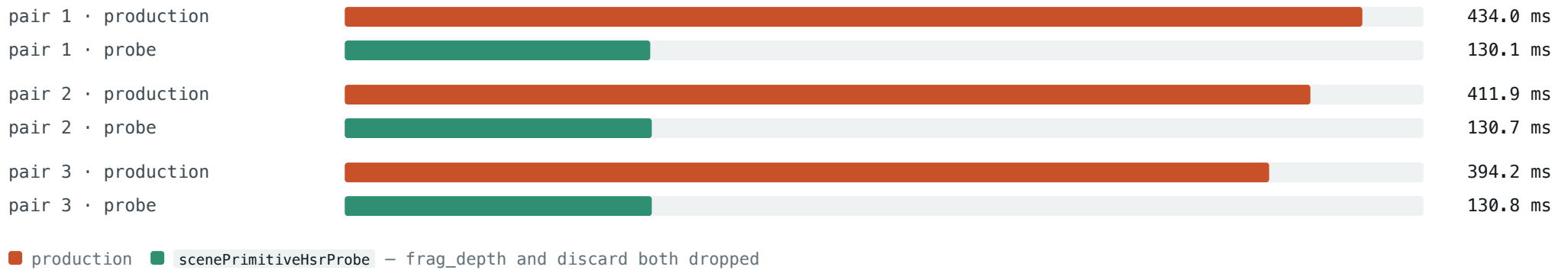
The brick raster is not exposed to this because it was redesigned around it. Its first pass writes no depth and shades nothing: it appends the instance index into a fixed 24-entry per-pixel arena with one `atomicAdd`, and its `discard` only suppresses an overflow-mask colour write *after* the append. A second full-screen pass then walks

each pixel's candidates front-to-back exactly once. Overflow beyond 24 candidates re-runs the historical direct fragment in an isolated pipeline, so capacity is a performance parameter and never changes the image.



77.8 % of the screen is covered by at least one proxy. Over covered pixels the mean is **8.95**, of which **8.78** are marched kinds; median 7, p90 20, p99 33, max 59. Tightening the proxy from the axis-aligned world box to the primitive's own oriented box halves this and was *refuted* in the same lane (−5.2, +3.7 and −24.1 ms against a control whose own spread was 356.7–385.8 ms): fragment *count* is not what this pass is bound by, per-fragment march cost is. Which is the same thing \$5.2's two maps say.

THE UPPER BOUND — INTERLEAVED, PAIRED, 3 PAIRS



Fresh process per arm, four warm-up frames, ten seconds of continuously submitted frames, median of each. Paired deltas **303.9 / 281.2 / 263.4 ms**; ratios **3.34 / 3.15 / 3.01×**, median **3.15×**. The probe arm is also far steadier — **130.1 / 130.7 / 130.8** against **434.0 / 411.9 / 394.2** — because with hidden-surface removal working, the pass stops absorbing whatever else the machine is doing. Subtracting the 58.5 ms the rest of the frame costs, the pass itself goes from **347.1 ms** to about **72 ms**.

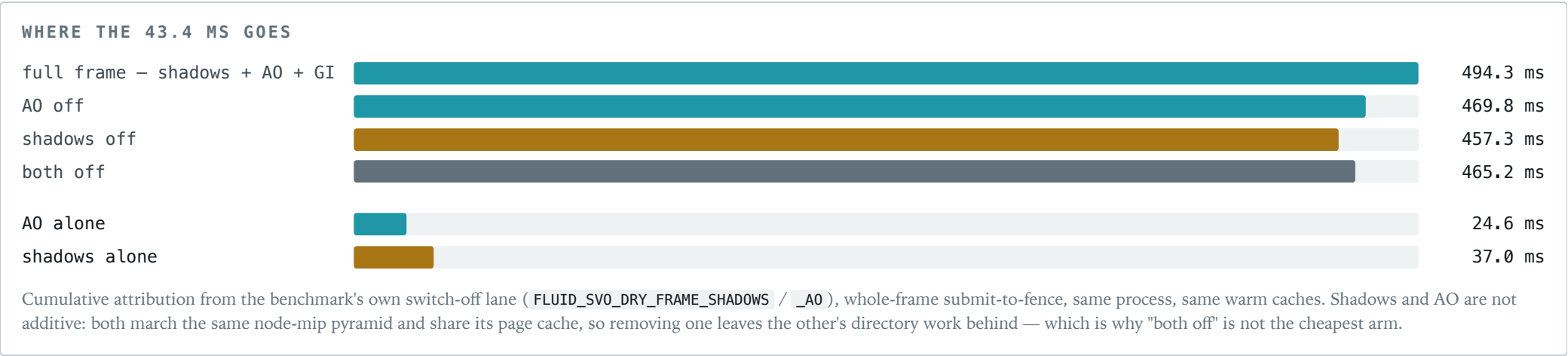
What this bounds, and what it does not — the probe over-states the fix. The probe's image is wrong in a specific way: with `frag_depth` gone, the depth the hardware tests and stores is the *proxy box's* back-face depth rather than the surface's, so it rejects proxies a correct resolve would still have to try. Part of that rejection is sound — a proxy wholly behind the ground the brick raster already wrote really is invisible — and part is not.

§5.3 separates the two, exactly, on **731** sampled covered pixels using the same CPU intersection oracle. A sound front-to-back resolve — entry-sorted, stopping once the next candidate begins behind the running best hit, and seeded with the ground depth — runs **5.22** marches per covered pixel against today's **9.11**. That is **1.75×** fewer field evaluations, not **3.15×**. Dropping the ground seed costs it a little: 5.76.

So the honest bracket is 1.75× to **3.15×**, and three things live in the gap that the march count does not capture: the resolve also shortens every march it does run to `[tEnter, best]` rather than the whole box span; it replaces 9.11 fragment-shader launches per pixel with one, plus a coverage append that costs what the brick raster's costs; and it removes the `discard` that is disabling hidden-surface removal for the entire pass today. The brick raster pays 0.95 ms and 1.88 ms for exactly this split, over a comparable instance count.

The second cost – cone visibility

With the wall removed, `Sparse voxel compact cone visibility` becomes the frame's largest pass. At 43.4 ms it is 10.7 % of the frame today and would be roughly a third of a repaired one, so it is worth stating what it is before it becomes the headline.



HOW IT WORKS

A cone is marched, not a ray. Step width follows the continuous cone diameter and each sample is a trilinear fetch from the node-mip opacity pyramid. Coverage is `C0`-continuous in `lod`: a single `floor(lod)` fetch made accumulated opacity jump wherever the level incremented, which rendered as concentric isodistance rings around point lights, so the two bracketing levels are blended — but only inside a trailing transition band, because full-range blending doubled per-step fetches and cost +35 % frame time.

The pass runs at half rate (800×620) over an 8×8 workgroup grid. Each lane reads the 2×2 full-resolution G-buffer footprint under it and tests it for homogeneity — same identity, depth within 1 %, normals within $1e-4$ of parallel. Homogeneous lanes resolve visibility for the whole quad; the rest enqueue their coordinate into an atomic boundary queue that a third dispatch in the same encoder retraces exactly. Reconstruction back to full rate is `full-res-relight`: the reduced plane carries only visibility, and material and BRDF are re-evaluated at full resolution in the deferred pass.

Measured reconstruction error against a full-rate reference over 1,984,000 lit pixels: mean relative luminance 1.81 %, p95 10.6 %, guided-upsample fallback band 0 % of hit pixels.

§ 8

What the profile says to do

1 • Give the scene-primitive raster the coverage/resolve treatment.

The design is already in the file, sixty lines above it: a coverage-only pass over the 501 proxies appending candidates with an `atomicAdd`, then one full-screen resolve that walks each pixel's list front-to-back and stops once the next candidate begins behind its best hit. Expect somewhere between $1.75\times$ — the exactly-measured reduction in field marches, 9.11 to 5.22 per covered pixel — and $3.15\times$, the hardware's own unsound upper bound. Two details differ from the brick case: primitive proxies overlap far more (median 7 against a brick's ~ 1), and the arena needs sizing for a p99 of 33 rather than 18, with the same exact overflow fallback catching the tail.

Seed the resolve's running best with the depth the brick raster already wrote. It is free — the buffer is loaded either way — and it is worth 0.54 marches per covered pixel on its own.

2 • Do not tighten the proxies.

The oriented-box variant halves mean covering proxies per pixel and measured -5.2 , $+3.7$ and -24.1 ms against a control whose own spread was 30 ms. The two maps in §5.2 say why: the coping owns 46 % of the coverage and 16 % of the cost, so removing coverage from the cheap objects removes nothing. This is a per-fragment-cost problem, not a fragment-count problem.

The corollary for §5.3: making the bonsai's fields cheaper is also the wrong lever *first*. Its 65 % share is 65 % of work that is mostly discarded.

3 • Then look at the cone pass, and only then.

43.4 ms is 10.7 % of this frame and would be roughly a third of a repaired one. Two levers already exist and are measured elsewhere: `coneLightingScale` 0.25 (quarter rate, currently in the performance preset) and `stationaryPrimaryReuseEnabled`, which skips primary visibility entirely while camera and scene are unchanged. The second is off by default and is *disabled outright* in the raster-primary path, because the rigid impostor pass blocks the reuse whenever the scene has bodies — this one has 5.

4 • The octree renderer itself is not the problem, and that is worth saying.

The complete sparse-voxel primary — cull, coverage, resolve, overflow, over 584 leaves and 299,008 voxels — costs **3.9 ms** at 1600×1240 , or 1.0 % of the frame. Its predecessor, the full-screen traversal megakernel, cost 27 ms on a comparable frame. That work landed; nothing in this profile argues with it. What this profile says is that the authored analytic set, which was never on that path, never got the same treatment.

The full pass table

Every channel of every labelled encoder, 33 steady-state frames. **p50** is the per-frame median; **min** and **max** are the extremes across those frames, not across intervals. Counters are means over uncontended samples attributed to that pass.

PASS	CH	P50 MS	MIN	MAX	% FRAME	FRAG	OCC	ALU	READ GB/S	WRITE GB/S	LLC
 exact live-scene primitive visibility	Fragment	346.22	219.84	439.11	85.35		21.1	22.4	1.2	11.2	8.5
 compact cone visibility	Compute	43.37	35.28	48.86	10.69		0.4	16.4	1.2	1.1	9.7
 deferred dry lighting	Fragment	10.66	10.54	11.49	2.63		12.0	10.9	20.2	45.7	28.8
 persistent world GI cache	Compute	2.35	2.34	2.86	0.58		0.1	58.0	2.1	0.7	17.5
 primary conservative coverage resolve	Fragment	1.83	1.82	2.09	0.45		18.5	40.8	2.0	12.8	5.4
 directional-light cache demand	Compute	1.16	0.896	1.56	0.29		5.3	12.0	14.7	1.0	12.5
 primary conservative brick coverage	Fragment	0.906	0.860	1.00	0.22		64.7	3.0	0.3	47.8	27.1
 exact live-scene primitive visibility	Vertex	0.901	0.073	0.986	0.22		21.1	22.4	1.2	11.2	8.5
 primary conservative coverage overflow	Vertex	0.841	0.648	0.931	0.21		64.6	9.4	39.6	48.6	31.0
 primary conservative coverage overflow	Fragment	0.243	0.228	0.365	0.060		64.6	9.4	39.6	48.6	31.0
 analytic rigid primary discovery	Fragment	0.108	0.093	0.414	0.027		22.9	18.2	18.3	40.6	23.2
 analytic rigid primary discovery	Vertex	0.070	0.018	0.263	0.017		22.9	18.2	18.3	40.6	23.2
 primary conservative coverage resolve	Vertex	0.047	0.040	0.070	0.012		18.5	40.8	2.0	12.8	5.4
 primary conservative brick coverage	Vertex	0.041	0.000	0.063	0.010		64.7	3.0	0.3	47.8	27.1
 raster-rigid certificate bridge	Fragment	0.036	0.030	0.123	0.009		39.6	44.3	16.8	18.3	10.9
Blit Command 10	Compute	0.035	0.026	0.185	0.009		—	—	—	—	—
 raster-rigid certificate bridge	Vertex	0.029	0.019	0.054	0.007		39.6	44.3	16.8	18.3	10.9
 raster thin-glass discovery	Vertex	0.029	0.016	0.050	0.007		12.9	32.2	0.2	0.8	3.1
 brick instance cull	Compute	0.028	0.021	0.045	0.007		10.7	10.0	17.4	41.3	27.0
Blit Command 2	Compute	0.027	0.024	0.045	0.007		—	—	—	—	—

PASS	CH	P50 MS	MIN	MAX	% FRAME	FRAG OCC	ALU	READ GB/S	WRITE GB/S	LLC
■ deferred dry lighting	Vertex	0.017	0.010	0.031	0.004	12.0	10.9	20.2	45.7	28.8
■ directional-light cache population	Compute	0.010	0.010	0.146	0.002	0.0	0.0	0.1	0.1	0.5
Blit Command 0	Compute	0.004	0.003	0.007	0.001	—	—	—	—	—
Blit Command 16	Compute	0.003	0.003	0.004	0.001	—	—	—	—	—
Blit Command 12	Compute	0.003	0.002	0.079	0.001	—	—	—	—	—
Blit Command 14	Compute	0.002	0.002	0.010	0.001	—	—	—	—	—

APPENDIX B

Reproducing this

```
# the capture – Metal System Trace + GPU counters
node --import tsx tools/profile-svo-render-xctrace.ts \
  --scene=hero-garden-hose --resolution=1600x1240 \
  --traversal=raster-primary --shading=split \
  --cone-tracing=cones --cone-scale=0.5 \
  --radiance-reconstruction=full-res-relight \
  --counter-seconds=5 --out=artifacts/xctrace-hero-garden-hose-2026-08-04

# the frame, its images, and the scene census
FLUID_SVO_DRY_FRAME_SCENE=hero-garden-hose FLUID_SVO_DRY_FRAME_WIDTH=1600 \
FLUID_SVO_DRY_FRAME_HEIGHT=1240 FLUID_SVO_DRY_FRAME_TRAVERSAL=raster-primary \
FLUID_SVO_DRY_FRAME_SHADING=split FLUID_WEBGPU_DAWN_FEATURES=skip_validation \
node --import tsx tools/benchmark-svo-dry-frame-gpu.ts

# the hidden-surface-removal upper bound, interleaved A B A B A B
FLUID_SVO_DRY_FRAME_SCENE_HSR_PROBE=1 # arm B only
FLUID_SVO_DRY_FRAME_PROFILE_SECONDS=10

# the two CPU censuses – no GPU. Coverage is a ray/AABB slab test of the
# camera's own pixel rays; the resolve order and every hit/miss above come
# from intersectSvoPrimitive, the CPU half of the WGSL intersection contract.
W=800 H=620 node --import tsx tmp/hero-proxy-coverage.ts # figure, histogram
W=800 H=620 node --import tsx tmp/hero-sdf-census.ts # fields, probes, resolve
```

All of this document's data lives in one file: `docs/svo-render-frame-anatomy.data.js`, regenerated by `tmp/hero-doc-data.mjs` from `artifacts/xctrace-hero-garden-hose-2026-08-04/` — the trace, the profiler's `summary.json`, the reduced `pass-table.json`, the two CPU censuses (`proxy-coverage.json`, `sdf-census.json`), the A/B log, and the benchmark's `frame/report.json`. The page states no number of its own; every figure and every sentence above binds one from that file.

Captured 2026-08-04 on Apple M1 Max, macOS 26.5.1 (25F80), Dawn/Metal 3. Source commit `a3adca9`. Companion to Anatomy of one simulation frame — symmetric-expansion.